Technical Architecture

cross-review-mcp v0.5.2 — System Design & Internals

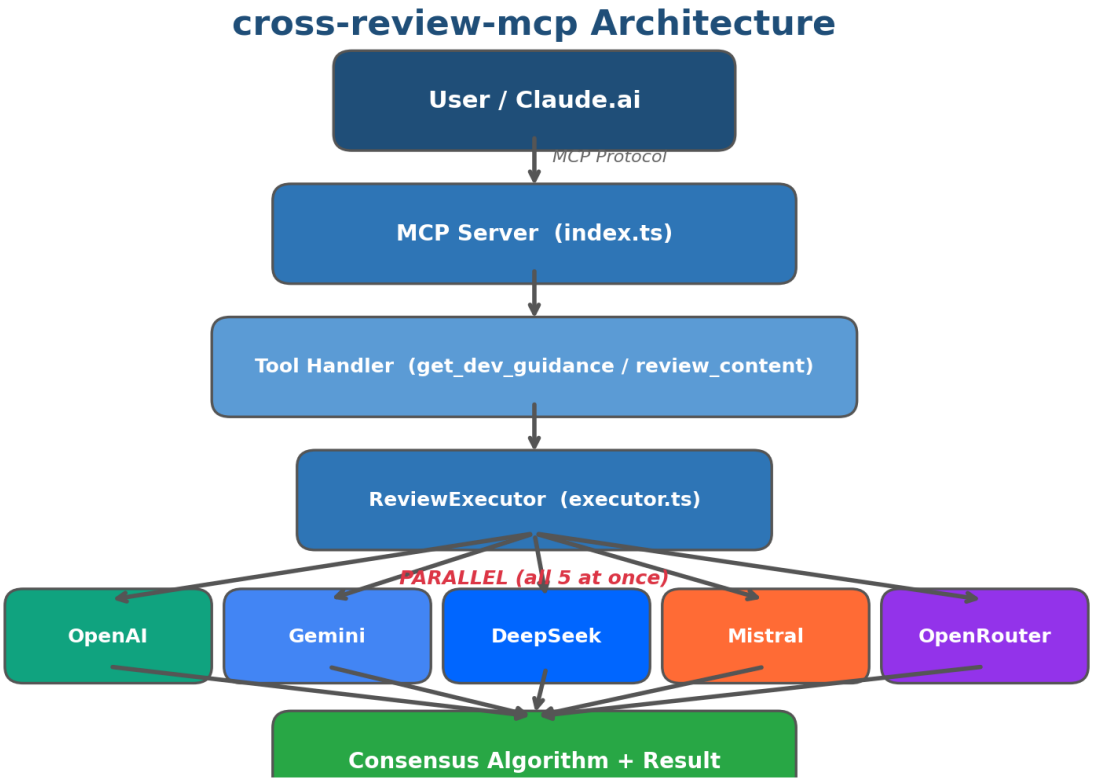

Figure 1: High-level architecture

System Components

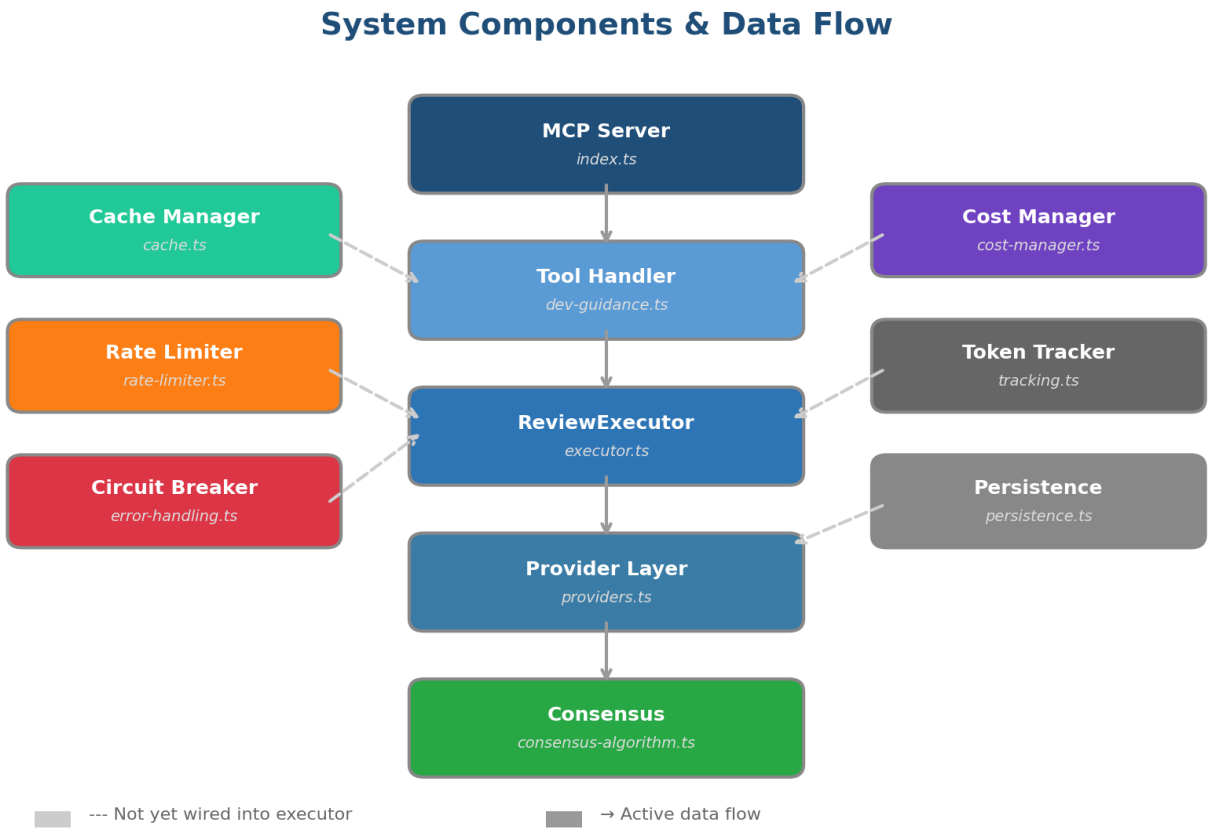

Figure 2: All components and integration status (dashed = not yet wired)

MCP Server (index.ts) — Entry point. Registers 4 MCP tools. Lazy-loads ReviewExecutor on first call. Suppresses console output to keep MCP protocol clean.

ReviewExecutor (executor.ts) — Core engine. Initializes provider connections. Sends requests to all 5 models via Promise.all(). Collects responses and calculates costs.

Provider Layer (providers.ts) — Unified interface for OpenAI, Gemini (Google AI SDK), and OpenAI-compatible APIs (DeepSeek, Mistral, OpenRouter).

Consensus Algorithm (consensus-algorithm.ts) — Analyzes 5 model responses. Finds majority diagnosis. Scores confidence. Detects paradigm shifts. Preserves divergent perspectives.

Dev Guidance (dev-guidance.ts) — Formats developer problems into structured prompts. Parses model responses. Returns actionable root cause + fix + 5 perspectives.

Cache Manager (cache.ts) — LRU/FIFO cache with SHA-256 content hashing, TTL expiry, disk persistence, hit/miss metrics. **Not yet wired into executor.**

Cost Manager (cost-manager.ts) — Tracks per-provider token usage and costs. Daily/monthly thresholds. Disk persistence. **Not yet wired into executor.**

Request Flow

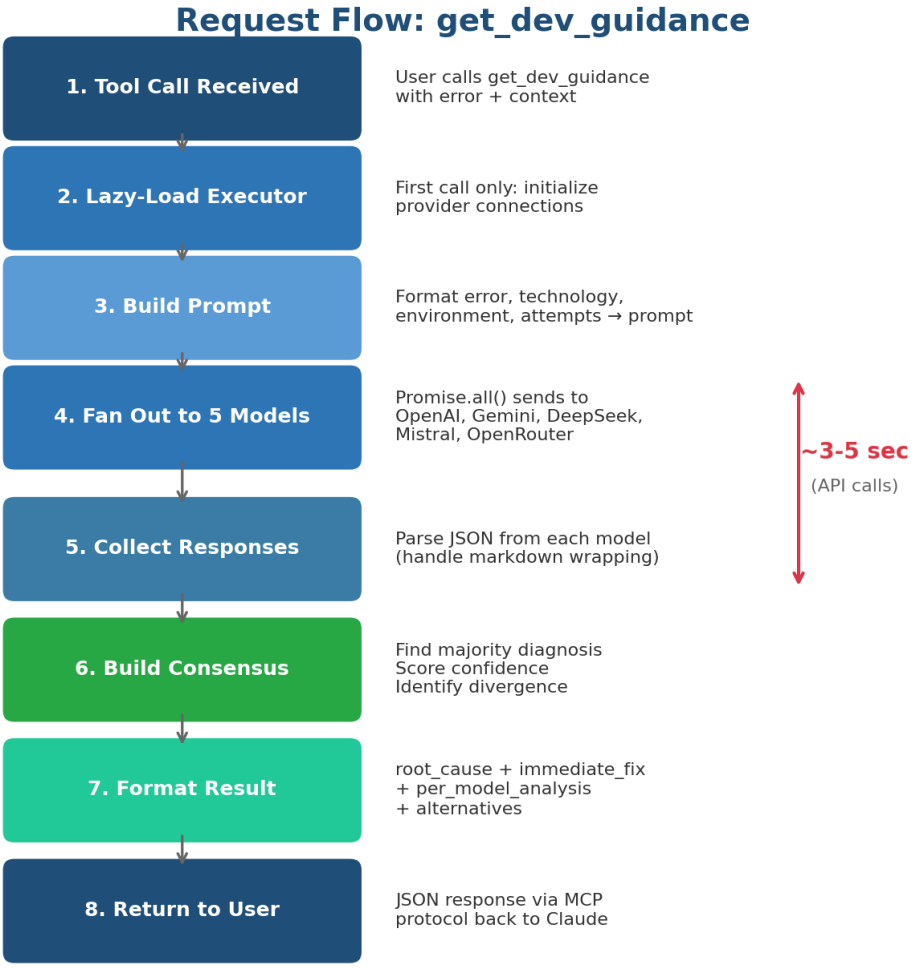

Figure 3: Step-by-step request flow with timing

Consensus Algorithm Deep Dive

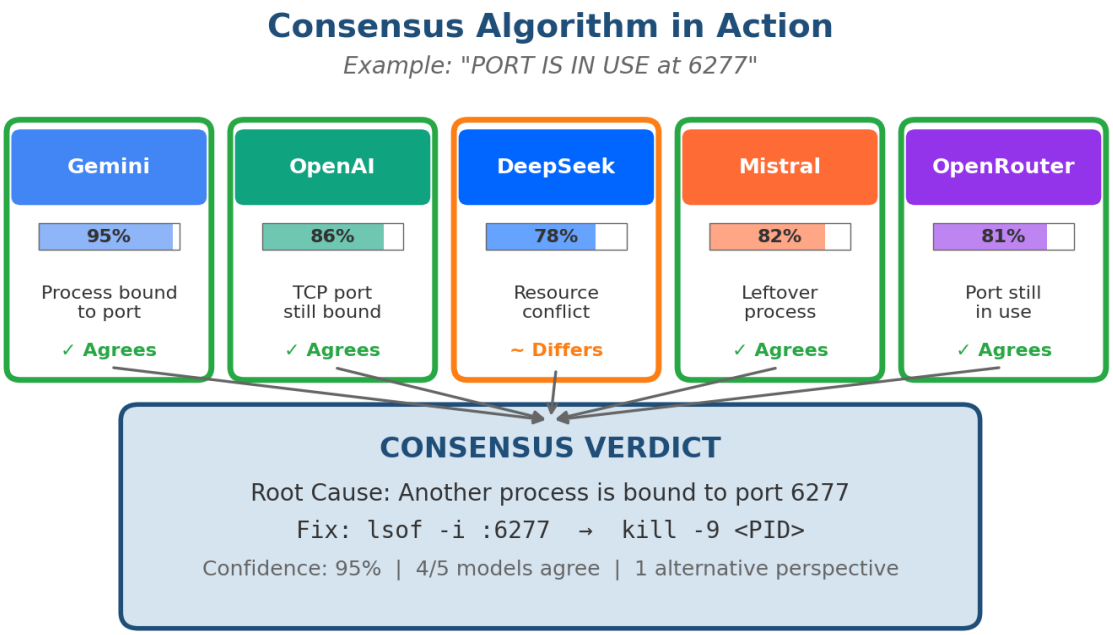

Figure 4: Consensus in action — agreement and divergence

Key innovation: Disagreement is preserved as "alternative perspectives" rather than being averaged away. When a problem is genuinely ambiguous, seeing 5 different angles helps developers think deeper.

Confidence formula: consensus_confidence = (agreeing_models / total_models). A 4/5 agreement yields 80%. Individual model confidence is self-reported (0-1).

Paradigm shift detection: When the top-2 diagnoses are fundamentally different, the algorithm flags this — indicating the problem may have multiple valid interpretations.

Performance & Cost

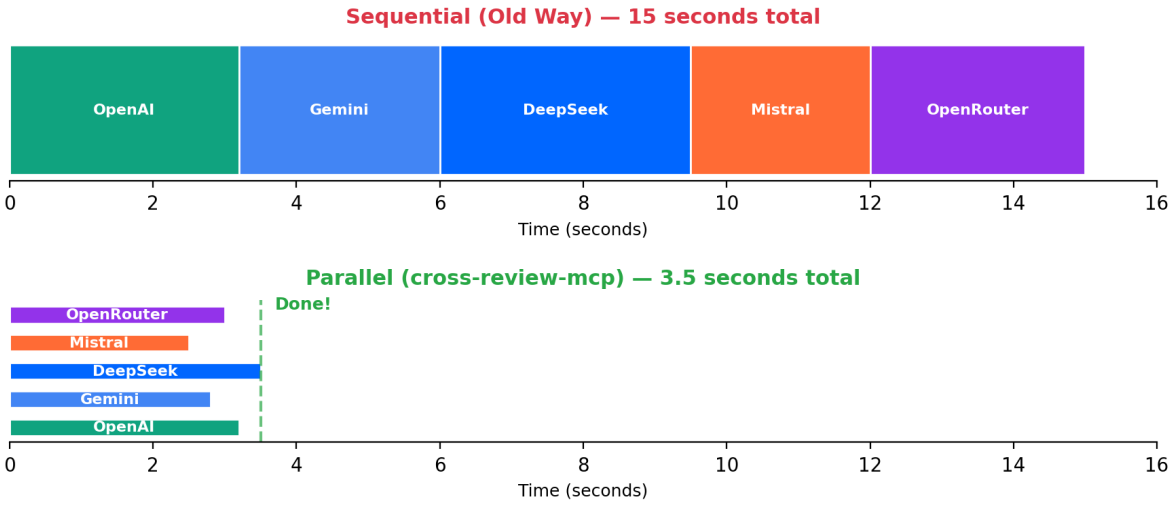

Figure 5: Parallel execution — same wall-clock time as single model

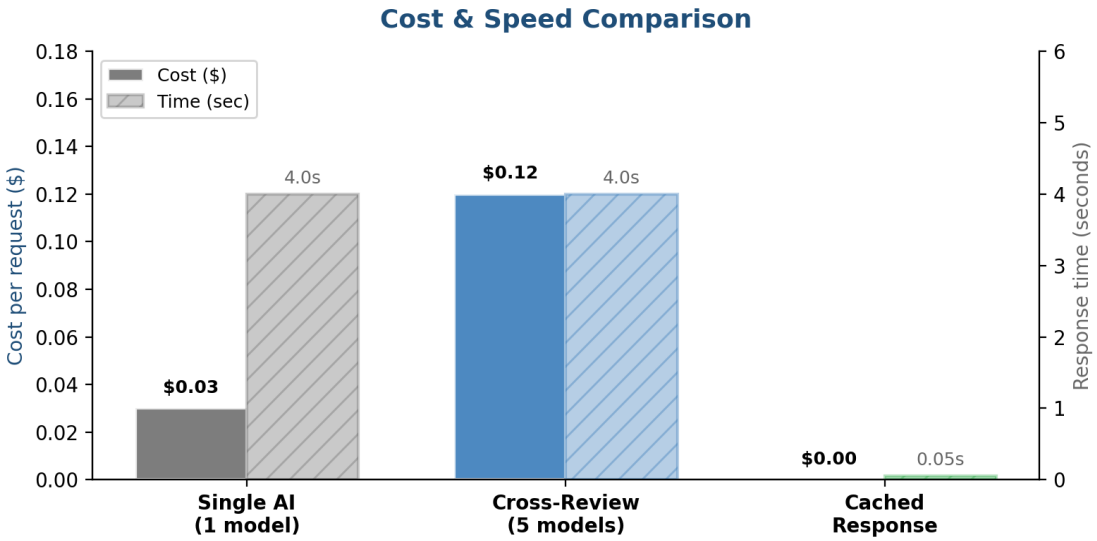

Figure 6: Cost per request — 5x models but same latency

Model Capability Comparison

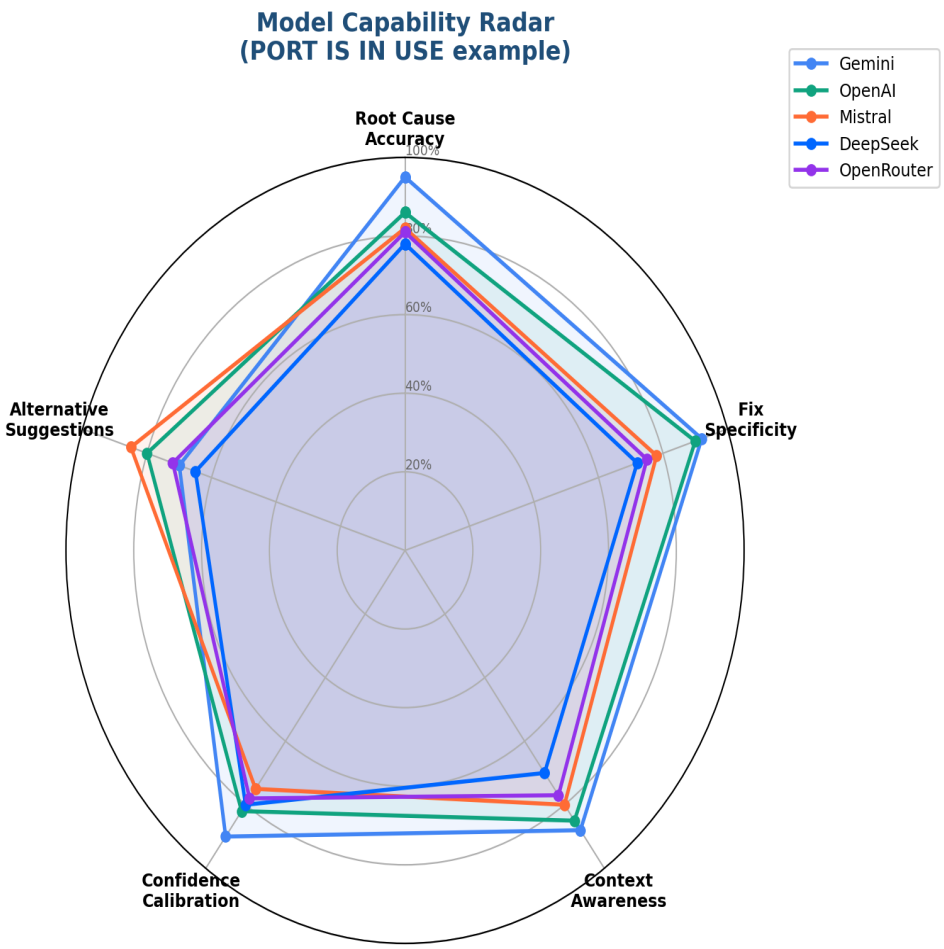

Figure 7: Model strengths across 5 capability axes

Integration Status

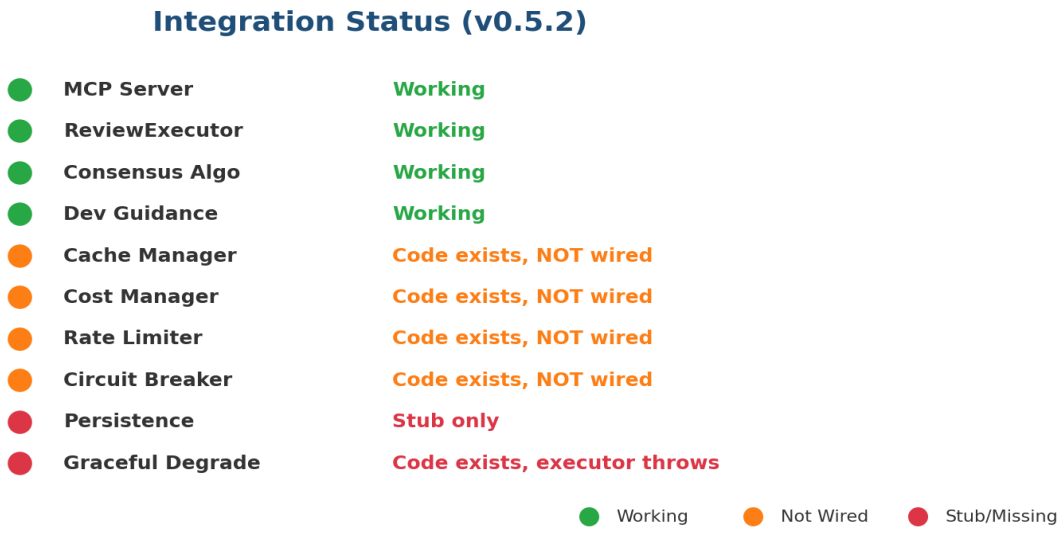

Figure 8: Module wiring status — several components exist but need integration

Critical gap: The ReviewExecutor is the central workhorse, but it does not use the cache, cost manager, rate limiter, or circuit breaker. These modules exist as standalone code but are not integrated into the execution pipeline. The executor also throws on missing API keys rather than gracefully skipping the provider.

Key Design Decisions

Parallel execution: All 5 models queried simultaneously via Promise.all(). Latency = max(individual), not sum.

Lazy loading: ReviewExecutor loaded on first tool call, not at server startup. Avoids API key errors on startup.

ESM throughout: .js extensions in imports, "type": "module" in package.json. No CommonJS.

Divergence-as-feature: Model disagreement is surfaced, not hidden. Multiple valid perspectives > forced consensus.

Console suppression: All console output suppressed before dotenv loads, to keep MCP stdio protocol clean.